



AtomQuest Hackathon 1.0

In-House Goal Setting & Tracking Portal — Full Build Instruction Manual
Stack: React + Node.js/Express + PostgreSQL · Free deployment on Railway + Vercel

1. Overview & Vision

We are building **GoalSync** — a three-role goal management portal that covers the full goal lifecycle: creation → approval → quarterly check-ins → analytics. The system is designed to be *audit-ready*, *real-time*, and *extensible*.

Design Philosophy

- **Role-scoped everything.** Every API route, every component, every redirect is decided by the user's role (employee / manager / admin). Never show an employee what belongs to a manager.
- **Locked goals are sacred.** Once a manager approves, no one edits without Admin creating an audit log entry first.
- **Compute scores server-side.** The four UoM formulas live in one utility file on the backend so they never drift between frontend display and export.
- **Progressive disclosure.** The UI shows only what the user can act on right now (e.g., no achievement form if the check-in window is closed).

User Roles at a Glance

Role	Key Capabilities	Cannot Do
Employee	Create/submit goals, log actuals, view own dashboard	Approve goals, view other employees

Manager (L1)	Approve/rework goals, conduct check-ins, view team	Create own goals (they're also employees — use both roles)
Admin / HR	Configure cycles, unlock goals, export reports, view org	Cannot submit check-ins or fake achievement data

 Important: Manager is also an Employee

A manager has their own goals too. The system assigns roles per user; a manager can switch context between "My Goals" (employee view) and "My Team" (manager view). Implement this as a role-context toggle in the navbar, not as separate accounts.

2. Technology Stack

Layer	Choice	Why
Frontend	React 18 + Vite	Fast dev server, small bundle, industry standard
Styling	Tailwind CSS	Utility-first, zero custom CSS bloat, dark mode trivial
State Mgmt	React Context + useReducer	No Redux overhead; sufficient for this app size
Charts	Recharts	React-native, composable, free
Date handling	date-fns	Lightweight, tree-shakeable
HTTP client	Axios	Interceptors for auth token injection
Backend	Node.js 20 + Express 4	JavaScript all the way, huge ecosystem
Database	PostgreSQL 16	ACID compliance, JSON support, audit log triggers

DB client	pg (node-postgres)	Lightweight, no ORM magic hiding SQL bugs
Auth	JWT (jsonwebtoken)	Stateless, easy role encoding in payload
Password hash	bcrypt	Industry standard
Cron jobs	node-cron	In-process scheduler for escalation checks
Email	Nodemailer + Mailtrap	Free during dev; swap SMTP for prod
Excel export	exceljs	Full xlsx with formatting, no license issues
CSV export	json2csv	Tiny, fast
Hosting (BE)	Railway.app	Free PostgreSQL + Node.js, GitHub deploy, no CC needed
Hosting (FE)	Vercel	Free tier, GitHub deploy, instant CDN

3. Architecture

Frontend (Vercel) — React SPA

Port 5173 (dev) → <https://goalsynce.vercel.app> (prod)

React Router v6 · Axios with JWT interceptor · Recharts analytics · Tailwind UI

↕ REST API (JSON)

Backend (Railway) — Node.js + Express

Port 4000 (dev) → <https://goalsynce-api.railway.app> (prod)

Auth middleware · Role guards · Business logic · Cron jobs · Email service

↕ SQL (pg)

PostgreSQL (Railway Add-on)

Free 1GB on Railway — same project as the backend

12 tables · DB-level audit trigger · Row-level security via app layer

Request Lifecycle

1. User action in React component → Axios call to `/api/...`
2. Express router matches path → Auth middleware validates JWT → Role guard checks permission
3. Controller calls service function (business logic) → Service queries PostgreSQL via pg pool
4. Audit logger writes change record if goal is locked
5. JSON response back to Axios → React state updates → UI re-renders

4. Complete File Structure

```
goalsynce/ # monorepo root |— client/ # React frontend |— public/ |
|   |— favicon.svg |— src/ | |— main.jsx # React entry: renders App
| |— App.jsx # Router + AuthProvider wrapper | |— context/ | |
|   |— AuthContext.jsx # JWT decode, user state, login/logout | |—
hooks/ | | |— useAuth.js # shortcut to useContext(AuthContext) | |
|— useGoals.js # goals fetch/mutate hooks | | |— useCycle.js #
active cycle state | |— services/ | | |— api.js # Axios instance +
interceptors | | |— authService.js # Login(), Logout(), me() | |
|— goalService.js # CRUD for goals | | |— achievementService.js #
quarterly actuals | | |— managerService.js # team view, approve,
checkin | | |— adminService.js # cycles, users, reports | |—
utils/ | | |— scoreCalc.js # UoM score formulas (mirrors server) | |
|— cycleUtils.js # which quarter is active now? | | |—
formatters.js # date, number, % formatters | |— components/ | | |—
common/ | | | |— Navbar.jsx # top nav with role-context switcher | |
| |— Sidebar.jsx # Left nav links per role | | | |—
ProtectedRoute.jsx # redirects to /login if not authenticated | | | |—
RoleGuard.jsx # renders children only if role matches | | | |—
LoadingSpinner.jsx | | | |— StatusBadge.jsx # Not Started / On Track
/ Completed | | | |— ProgressBar.jsx # visual score bar | | | |—
Modal.jsx # generic modal wrapper | | | |— Toast.jsx # success/error
toasts | | | |— ExportButton.jsx # CSV / Excel download trigger | |
|— employee/ | | | |— GoalForm.jsx # create/edit goal with
validation | | | |— GoalCard.jsx # summary card for one goal | | |
|— GoalSheet.jsx # full goal sheet view (all goals) | | | |—
AchievementForm.jsx # quarterly actual entry | | | |—
WeightageGauge.jsx # visual 100% gauge | | | |— manager/ | | | |—
```

```

TeamTable.jsx # list of team members + status | | | | └─
ApprovalPanel.jsx # inline edit + approve/return | | | | └─
CheckinModal.jsx # quarterly checkin comment form | | | | └─
TeamProgressChart.jsx # bar chart of team scores | | | └─ admin/ | | |
└─ CycleManager.jsx # create/edit goal cycles | | | └─ UserManager.jsx
# user list, role assign, hierarchy | | | └─ SharedGoalPusher.jsx #
push KPI to multiple employees | | | └─ UnlockGoal.jsx # exception
handling with audit log | | | └─ CompletionDashboard.jsx # checkin
completion heatmap | | | └─ AuditLogTable.jsx # audit trail viewer | |
└─ pages/ | | | └─ Login.jsx | | | └─ employee/ | | | └─ Dashboard.jsx #
my goals overview | | | └─ GoalsPage.jsx # full goal sheet | | | └─
CheckinPage.jsx # quarterly update | | | └─ manager/ | | | └─
TeamDashboard.jsx # team overview | | | └─ ApprovalQueue.jsx # pending
approvals list | | | └─ CheckinView.jsx # team quarterly progress | |
└─ admin/ | | | └─ AdminDashboard.jsx | | | └─ Reports.jsx | | | └─
Settings.jsx | | | └─ index.html | | | └─ vite.config.js | | | └─
tailwind.config.js | | | └─ package.json | | | └─ server/ # Node.js backend |
└─ src/ | | | └─ app.js # Express setup, middlewares, routes mount | |
└─ server.js # process.listen + DB connect + cron start | | | └─ config/
| | | └─ db.js # pg Pool, export query() helper | | | └─ middleware/ | |
| | | └─ authenticate.js # verify JWT, attach req.user | | | └─
authorize.js # role whitelist: authorize('admin', 'manager') | | | └─
routes/ | | | | └─ auth.routes.js | | | | └─ goals.routes.js | | | | └─
achievements.routes.js | | | | └─ manager.routes.js | | | | └─
admin.routes.js | | | | └─ reports.routes.js | | | | └─ controllers/ | | |
└─ auth.controller.js | | | | └─ goals.controller.js | | | | └─
achievements.controller.js | | | | └─ manager.controller.js | | | | └─
admin.controller.js | | | | └─ reports.controller.js | | | | └─ services/
| | | | | └─ goal.service.js # validation, weightage check, lock logic | | |
└─ score.service.js # UoM score formulas | | | | └─ cycle.service.js #
which window is active now | | | | └─ email.service.js # Nodemailer
wrappers | | | | └─ audit.service.js # writeAuditLog() | | | | └─
escalation.service.js # escalation rule evaluation | | | | └─ jobs/ | | |
└─ escalation.job.js # node-cron: runs daily at 8am | | | | └─ utils/ | | |
└─ response.js # sendSuccess(), sendError() helpers | | | | └─ migrations/
| | | | | └─ 001_create_users.sql | | | | | └─ 002_create_cycles.sql | | |
003_create_thrust_areas.sql | | | | | └─ 004_create_goals.sql | | | |
005_create_achievements.sql | | | | | └─ 006_create_checkins.sql | | |
007_create_audit_logs.sql | | | | | └─ 008_create_notifications.sql | | |
009_create_escalation_rules.sql | | | | | └─ 010_seed_demo_data.sql | | |

```

.env.example | package.json | .gitignore | README.md |
architecture.png

5. Database Schema

All tables use `UUID` primary keys generated with `gen_random_uuid()`. Every table that represents user content has `created_at` and `updated_at` timestamps. Run all `migrations/` files in order.

Table: users

```
CREATE TABLE users (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  name        VARCHAR(120) NOT NULL,  
  email       VARCHAR(200) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  role        VARCHAR(20) NOT NULL CHECK (role IN ('employee', 'manager', 'admin')),  
  department  VARCHAR(100),  
  manager_id  UUID REFERENCES users(id) ON DELETE SET NULL, -- L1  
  is_active   BOOLEAN DEFAULT TRUE,  
  created_at  TIMESTAMPTZ DEFAULT NOW(),  
  updated_at  TIMESTAMPTZ DEFAULT NOW()  
);
```

Note on manager_id: An employee has `manager_id` pointing to their L1 manager. The manager's own `manager_id` points to their skip-level (L2). Admin users can have `manager_id = NULL`.

Table: goal_cycles

```
CREATE TABLE goal_cycles (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  name        VARCHAR(100) NOT NULL, -- e.g. "FY 2025-26"  
  year        INTEGER NOT NULL,
```

```

    phasel_start    DATE NOT NULL,           -- Goal setting opens
    phasel_end      DATE NOT NULL,           -- Goal setting closes
    q1_start        DATE NOT NULL,           -- July
    q1_end          DATE NOT NULL,
    q2_start        DATE NOT NULL,           -- October
    q2_end          DATE NOT NULL,
    q3_start        DATE NOT NULL,           -- January
    q3_end          DATE NOT NULL,
    q4_start        DATE NOT NULL,           -- March/April
    q4_end          DATE NOT NULL,
    is_active       BOOLEAN DEFAULT FALSE,
    created_by      UUID REFERENCES users(id),
    created_at      TIMESTAMPTZ DEFAULT NOW()
);

```

Table: thrust_areas

```

CREATE TABLE thrust_areas (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name          VARCHAR(150) NOT NULL,
    description    TEXT,
    cycle_id      UUID REFERENCES goal_cycles(id) ON DELETE CASCADE,
    created_by    UUID REFERENCES users(id),
    is_active     BOOLEAN DEFAULT TRUE
);

```

Table: goals

```

CREATE TABLE goals (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    employee_id   UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    cycle_id      UUID NOT NULL REFERENCES goal_cycles(id) ON DELETE CASCADE,
    thrust_area_id UUID REFERENCES thrust_areas(id),
    title         VARCHAR(255) NOT NULL,
    description    TEXT,
    uom_type      VARCHAR(20) NOT NULL CHECK (uom_type IN ('min', 'max')),
    -- min = higher is better (sales), max = lower is better (cost), t
    target_value  NUMERIC,                  -- numeric target (NUMERIC)
    target_date   DATE,                    -- for timeline UoM
    weightage     NUMERIC(5,2) NOT NULL,    -- e.g. 20.00
    status        VARCHAR(20) DEFAULT 'draft'
);

```

```

        CHECK (status IN ('draft','submitted','approved','
is_locked      BOOLEAN DEFAULT FALSE,          -- TRUE after manager
is_shared      BOOLEAN DEFAULT FALSE,          -- pushed from admin/n
parent_goal_id UUID REFERENCES goals(id) ON DELETE SET NULL, --
created_at     TIMESTAMPTZ DEFAULT NOW(),
updated_at     TIMESTAMPTZ DEFAULT NOW()
);
```

Critical constraint enforced in app layer (not DB):

- Max 8 goals per employee per cycle
- Total weightage must equal 100% on submission
- Min weightage per goal: 10%
- Shared goal recipients: weightage editable, title/target read-only

These are enforced in `goal.service.js → validateGoalSheet()` before any INSERT.

Table: goal_achievements

```

CREATE TABLE goal_achievements (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  goal_id     UUID NOT NULL REFERENCES goals(id) ON DELETE CASCADE,
  quarter     VARCHAR(5) NOT NULL CHECK (quarter IN ('Q1','Q2','Q3','Q4')),
  planned_value NUMERIC,          -- what was set as target
  actual_value  NUMERIC,          -- what employee logged
  completion_date DATE,           -- for timeline UoM
  goal_status   VARCHAR(20) DEFAULT 'not_started'
              CHECK (goal_status IN ('not_started','on_track','completed')),
  submitted_at  TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE (goal_id, quarter)       -- one record per goal
);
```

Table: manager_checkins


```
CREATE TABLE manager_checkins (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  goal_id     UUID NOT NULL REFERENCES goals(id) ON DELETE CASCADE,
  manager_id  UUID NOT NULL REFERENCES users(id),
  quarter     VARCHAR(5) NOT NULL CHECK (quarter IN ('Q1','Q2','Q3'),
  comment     TEXT NOT NULL,
  created_at  TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE (goal_id, manager_id, quarter)
);
```

Table: audit_logs

```
CREATE TABLE audit_logs (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  goal_id     UUID REFERENCES goals(id) ON DELETE SET NULL,
  user_id     UUID NOT NULL REFERENCES users(id),
  action      VARCHAR(100) NOT NULL,          -- e.g. 'goal_unlocked'
  field_name  VARCHAR(100),                  -- which field changed
  old_value   TEXT,
  new_value   TEXT,
  ip_address  VARCHAR(50),
  created_at  TIMESTAMPTZ DEFAULT NOW()
);
```

Table: notifications

```
CREATE TABLE notifications (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  title       VARCHAR(200) NOT NULL,
  message     TEXT NOT NULL,
  link        VARCHAR(500),                  -- deep link in app
  is_read     BOOLEAN DEFAULT FALSE,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);
```

Table: escalation_rules Bonus

```
CREATE TABLE escalation_rules (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  rule_type   VARCHAR(50) NOT NULL,
  -- 'goal_not_submitted' | 'goal_not_approved' | 'checkin_not_done'
  days_threshold INTEGER NOT NULL DEFAULT 7,
  is_active   BOOLEAN DEFAULT TRUE,
  created_by  UUID REFERENCES users(id),
  created_at  TIMESTAMPTZ DEFAULT NOW()
);
```

Table: escalation_logs Bonus

```
CREATE TABLE escalation_logs (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  rule_id     UUID REFERENCES escalation_rules(id),
  target_user_id UUID REFERENCES users(id),
  notified_user_id UUID REFERENCES users(id), -- who was notified
  reason      TEXT,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);
```

6. API Design

All routes prefixed with `/api`. All responses follow: `{ success: bool, data: any, message: string }`. Protected routes require `Authorization: Bearer <token>` header.

Auth Routes — `/api/auth`

Method	Path	Auth?	Body / Notes
POST	/login	No	<code>{ email, password }</code> → returns JWT + user object

GET	/me	Yes	Returns current user profile from token
POST	/logout	Yes	Client-side only (JWT is stateless); server logs event

JWT Payload: `{ id, name, email, role, department, manager_id, iat, exp }`. Set expiry to 8 hours. Store in `localStorage` on the client (note: for a hackathon this is fine; production would use `httpOnly` cookies).

Goals Routes — `/api/goals`

Method	Path	Role	Description
GET	/	Employee	Get my goals for active cycle. Query: <code>? cycle_id=</code>
POST	/	Employee	Create a new goal (draft). Validates max 8 goals.
PUT	/:id	Employee	Edit goal (only if status = draft or rework)
DELETE	/:id	Employee	Delete goal (only if draft, not submitted)
POST	/submit	Employee	Submit all draft goals. Validates weightage = 100%, min 10% each.
GET	/:id	Employee/Mgr	Get single goal with all achievements

Achievements Routes — `/api/achievements`

Method	Path	Role	Description
GET	/:goal_id	Employee	Get all quarterly achievements for a goal
POST	/	Employee	Submit quarterly actual. Body: <code>{ goal_id, quarter, actual_value, goal_status }</code> . Validates quarter window is open.
PUT	/:id	Employee	Update existing actual (within open window)

Manager Routes — `/api/manager`

Method	Path	Description
GET	/team	Get all employees under this manager with their goal submission status
GET	/team/:employee_id/goals	Get full goal sheet of a specific employee
PUT	/goals/:id/approve	Approve a goal. Locks the goal. Sets status = approved. Sends notification.
PUT	/goals/:id/rework	Return for rework with a comment. Sets status = rework.
PUT	/goals/:id/edit	Inline edit target/weightage during approval flow (before approve click).
POST	/checkin	Submit check-in comment. Body: <code>{ goal_id, quarter, comment }</code> .
GET	/checkin/:employee_id/:quarter	Get all check-in comments for an employee in a quarter

Admin Routes — `/api/admin`

Method	Path	Description
GET/POST/PUT	/cycles	CRUD for goal cycles. Only one active cycle at a time.
GET/POST/PUT	/thrust-areas	Manage thrust areas per cycle
GET/POST/PUT	/users	Create users, assign roles, set manager hierarchy
POST	/shared-goals	Push a KPI goal to multiple employees. Body: <code>{ goal_template, employee_ids }</code>
POST	/goals/:id/unlock	Unlock a locked goal for editing. Requires reason. Writes audit log.
GET	/completion	Real-time completion dashboard: who submitted goals, who did checkins

GET	/audit-logs	Paginated audit log with filters
-----	-------------	----------------------------------

Reports Routes — `/api/reports`

Method	Path	Description
GET	/achievement	Planned vs actual for all employees. Query: <code>?format=csv excel&cycle_id=</code>
GET	/completion-rate	Checkin completion % per manager
GET	/goal-distribution	Goals by thrust area, UoM type, status

7. Business Logic (Critical Rules)

7.1 Goal Submission Validation — `goal.service.js` →

`validateGoalSheet()`

```
// This function runs BEFORE setting status to 'submitted'
async function validateGoalSheet(employeeId, cycleId) {
  const goals = await getGoalsByEmployee(employeeId, cycleId);

  if (goals.length === 0) throw new Error('No goals to submit');
  if (goals.length > 8) throw new Error('Maximum 8 goals allowed');

  const totalWeightage = goals.reduce((sum, g) => sum + parseFloat(g.weightage), 0);
  if (Math.abs(totalWeightage - 100) > 0.01) {
    throw new Error(`Total weightage must be 100%. Currently: ${totalWeightage}`);
  }

  const underMin = goals.filter(g => parseFloat(g.weightage) < 10);
  if (underMin.length > 0) {
    throw new Error(`Goals must have minimum 10% weightage: ${underMin.length} goals`);
  }
}
```

```
    return true;
}
```

7.2 Goal Lock Flow

1. Employee submits → all goals change to `status='submitted'`
2. Manager approves a specific goal → `status='approved', is_locked=TRUE`
3. Manager returns for rework → `status='rework'` → employee can edit → resubmit
4. Admin unlock → `is_locked=FALSE, status='approved'` + mandatory audit log entry
5. After all goals approved → send email to employee: "Your goals have been finalized"

7.3 Shared Goals

```
// admin.controller.js → pushSharedGoal()
// 1. Admin creates a "template" goal record (is_shared=TRUE, no emp
// 2. For each target employee, create a copy:
//    - title, target_value, uom_type from template (read-only for r
//    - is_shared = TRUE
//    - parent_goal_id = template goal's id
//    - weightage: employee can set this (must still comply with 10%
// 3. When primary owner updates achievement → cascade update to all
//    (run a query: UPDATE goal_achievements WHERE goal_id IN
//    (SELECT id FROM goals WHERE parent_goal_id = :parentId))
```



7.4 Cycle Window Check — `cycle.service.js` →

`getActiveWindow()`

```
// Returns: 'phase1' | 'q1' | 'q2' | 'q3' | 'q4' | null
function getActiveWindow(cycle) {
    const now = new Date();
    if (now >= cycle.phase1_start && now <= cycle.phase1_end) return '
    if (now >= cycle.q1_start && now <= cycle.q1_end) return 'q1';
```

```

    if (now >= cycle.q2_start && now <= cycle.q2_end) return 'q2';
    if (now >= cycle.q3_start && now <= cycle.q3_end) return 'q3';
    if (now >= cycle.q4_start && now <= cycle.q4_end) return 'q4';
    return null;
}
// Achievement submission is BLOCKED if getActiveWindow() returns null
// The frontend shows a "Check-in window is closed" message

```

8. Score Calculation — `score.service.js`

This is the most important calculation in the system. Define it ONCE here, and import it everywhere (both server-side for reports and copy to

`client/src/utils/scoreCalc.js`).

```

/**
 * Returns a score between 0 and 1 (or higher if overachievement)
 * @param {string} uomType - 'min' | 'max' | 'timeline' | 'zero'
 * @param {number} target
 * @param {number} actual
 * @param {Date} targetDate - for timeline UoM
 * @param {Date} completionDate - for timeline UoM
 */
function calculateScore(uomType, target, actual, targetDate, completionDate) {
  switch (uomType) {
    case 'min':
      // Higher is better (Sales Revenue)
      // Formula: Achievement ÷ Target
      if (!target || target === 0) return 0;
      return actual / target; // e.g. 90/100 = 0.9 → 90%

    case 'max':
      // Lower is better (TAT, Cost)
      // Formula: Target ÷ Achievement
      if (!actual || actual === 0) return 1; // achieved zero = 100%
      return target / actual; // e.g. 100/80 = 1.25 → 125% (overachievement)

    case 'timeline':
      // Date-based completion
      // If completed on or before deadline → 100%, else proportionate
      if (!completionDate) return 0;
      const deadlineMs = new Date(targetDate).getTime();
      const completedMs = new Date(completionDate).getTime();

```

```
    if (completedMs <= deadlineMs) return 1; // on time = 100%
    // Late: penalize proportionally (simple: return 0.5 for late)
    return 0.5;

    case 'zero':
      // Zero = Success (Safety incidents, defects)
      return actual === 0 ? 1 : 0;

    default:
      return 0;
  }
}

// Convert score to display %:
// displayScore = Math.min(calculateScore(...) * 100, 150).toFixed(1)
// Cap display at 150% to handle overachievement gracefully
```

Weighted Overall Score:

```
overallScore = goals.reduce((sum, goal) => {
  const score = calculateScore(goal.uom_type, ...);
  return sum + (score * (goal.weightage / 100));
}, 0);
// This gives a weighted average score (0 to 1+)
```

9. Validation Rules (Frontend & Backend)

Always validate on BOTH frontend and backend. Frontend for UX, backend for security.

Goal Form Validations

Field	Rule	Error Message
Title	Required, 3-255 chars	"Goal title is required"

Thrust Area	Required (select from list)	"Please select a thrust area"
UoM Type	Required, one of: min/max/timeline/zero	"Select a unit of measurement"
Target Value	Required if UoM = min or max. Positive number.	"Target must be a positive number"
Target Date	Required if UoM = timeline. Must be in future.	"Target date must be in the future"
Weightage	Min 10, Max 90, increments of any decimal	"Minimum weightage is 10%"
Goal Count	Max 8 goals per employee per cycle	"Maximum 8 goals allowed per cycle"
Total Weightage	Must = 100% on submit	"Total weightage is X%. Must be exactly 100%."

The WeightageGauge Component

Build a real-time visual gauge that shows current total weightage as the employee adds/edits goals. It should show:

- A horizontal bar filling from 0 to 100%
- Green when exactly 100%
- Red when over 100%
- Orange when under 100%
- Text showing "Used: X% / Remaining: Y%"

10. Frontend Pages — Detailed Spec

Employee Pages

Login Page (`Login.jsx`)

- Email + Password fields

- On success: decode JWT role → redirect to `/employee`, `/manager`, or `/admin`
- Show demo credential buttons for judges: "Login as Employee | Manager | Admin"

Employee Dashboard (`employee/Dashboard.jsx`)

- Header: "My Goals — FY 2025-26"
- 4 stat cards: Total Goals, Overall Score %, Goals Approved, Pending Check-ins
- List of GoalCards — each shows: Title, Thrust Area, Weightage %, Score %, Status badge
- Prominent "Add Goal" button (disabled if goals are locked or window closed)
- Banner: current window status ("Goal Setting Open" / "Q1 Check-in Open" / "Window Closed")

Goal Sheet (`employee/GoalsPage.jsx`)

- Table view of all goals with all columns
- Inline WeightageGauge at bottom
- "Submit Goals" button (validates and calls POST `/api/goals/submit`)
- Status column: Draft / Submitted / Approved / Rework (color coded)
- Rework goals show manager's comment in an alert box

Check-in Page (`employee/CheckinPage.jsx`)

- Quarter selector (Q1/Q2/Q3/Q4) — active quarter auto-selected
- For each goal: Planned Target | Actual input field | Status dropdown | Computed Score
- Score updates in real-time as employee types their actual
- Submit button saves all actuals

Manager Pages

Team Dashboard (`manager/TeamDashboard.jsx`)

- Team member list with: Name | Goals Submitted? | Approved? | Last Check-in
- Color-coded row: green=complete, orange=pending, red=missing

- Click on employee → navigate to their goal sheet
- Bar chart: team member names on X axis, overall score % on Y axis

Approval Queue (`manager/ApprovalQueue.jsx`)

- List of employees with submitted goals awaiting approval
- Expand employee → see all their goals in a table
- Each goal row: inline editable Target and Weightage fields
- Per-goal: "Approve"  or "Return for Rework"  buttons
- Rework requires a mandatory comment field
- "Approve All" button to bulk approve all goals of one employee

Check-in View (`manager/CheckinView.jsx`)

- Employee selector
- Quarter selector
- Shows: Goal Title | Planned | Actual | Score | Status | Manager Comment
- "Add Check-in Comment" button → opens CheckinModal

Admin Pages

Admin Dashboard (`admin/AdminDashboard.jsx`)

- Organization-wide metrics: Total Employees | Goals Submitted % | Check-ins Done %
- Completion heatmap: rows = departments, columns = quarters, cells = completion %
- Escalation log list (employees with no action)

Cycle Manager (`admin/Settings.jsx` → `CycleManager` tab)

- Form to create a new cycle: Name, Year, all window dates
- Toggle "Set as Active Cycle" (deactivates all others)
- Thrust area management: add/edit/delete areas per cycle

Reports Page (`admin/Reports.jsx`)

- Filters: cycle, department, status

- Achievement Report table: Employee | Goal | Planned | Actual | Score | Status
- Export buttons: "Download CSV" and "Download Excel"
- Completion Dashboard: which managers/employees have completed check-ins

11. State Management & Context

AuthContext (`context/AuthContext.jsx`)

```
// State shape:
{
  user: { id, name, email, role, department, manager_id } | null,
  token: string | null,
  isLoading: boolean,
  isAuthenticated: boolean,
}

// Actions:
login(email, password) // calls POST /api/auth/login, stores token
logout()               // clears localStorage, redirects to /login
refreshUser()          // calls GET /api/auth/me, refreshes user s

// HOW TO USE IN COMPONENTS:
const { user, isAuthenticated, login, logout } = useAuth();
// useAuth() is a custom hook in hooks/useAuth.js that wraps useCont
```

Axios Interceptor (`services/api.js`)

```
import axios from 'axios';

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:4000/api'
});

// Attach JWT to every request automatically
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('goalsynce_token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

```

    });

    // Handle 401 globally → logout and redirect
    api.interceptors.response.use(
      (res) => res,
      (err) => {
        if (err.response?.status === 401) {
          localStorage.removeItem('goalsynce_token');
          window.location.href = '/login';
        }
        return Promise.reject(err);
      }
    );

    export default api;

```

React Router Setup (`App.jsx`)

// Route structure:	
/login	→ Login.jsx (public)
/employee	→ ProtectedRoute → Employee Dashboard
/employee/goals	→ ProtectedRoute → GoalsPage
/employee/checkin	→ ProtectedRoute → CheckinPage
/manager	→ ProtectedRoute + RoleGuard(manager)
/manager/approvals	→ ApprovalQueue
/manager/checkins	→ CheckinView
/admin	→ ProtectedRoute + RoleGuard(admin)
/admin/reports	→ Reports
/admin/settings	→ Settings (cycles, users, thrust and more)
*	→ redirect to role-appropriate dashboard



12. Bonus Features — Implementation Guide

12.1 Email Notifications Bonus

Use **Nodemailer** with **Mailtrap** (free dev SMTP) or Gmail SMTP for demo.

Trigger Points (in controllers):

Event	Who Gets Email	Subject
Employee submits goals	Manager	"[GoalSync] Team member has submitted goals for review"
Manager approves	Employee	"[GoalSync] Your goals have been approved  "
Manager returns for rework	Employee	"[GoalSync] Goals returned for revision"
Q1 window opens	All employees	"[GoalSync] Q1 Check-in window is now open"
Escalation triggered	Manager / HR	"[GoalSync]  Action Required: Goal deadline missed"

12.2 Escalation Module Bonus

In `jobs/escalation.job.js`, schedule a daily cron at 8 AM:

```
const cron = require('node-cron');
cron.schedule('0 8 * * *', async () => {
  const rules = await getActiveRules();
  for (const rule of rules) {
    if (rule.rule_type === 'goal_not_submitted') {
      // Find employees who haven't submitted N days after phase1_st
      // Send reminder email to employee, then manager after 2xN day
    }
    if (rule.rule_type === 'goal_not_approved') {
      // Find managers with pending approvals older than N days
      // Notify manager, then their skip-level
    }
    if (rule.rule_type === 'checkin_not_done') {
      // Find employees with no achievement entry in current open q1
      // Notify employee, then manager
    }
    // Log to escalation_logs table
  }
});
```

12.3 Analytics Module Bonus

Build these charts using **Recharts**:

- **QoQ Trend:** LineChart with quarters on X axis, avg score on Y axis, one line per employee/team
- **Completion Heatmap:** Custom SVG grid — departments as rows, quarters as columns, color intensity = completion %
- **Goal Distribution:** PieChart by thrust area; BarChart by UoM type
- **Manager Effectiveness:** BarChart of check-in completion % per L1 manager

13. Free Deployment Guide

Step 1: Set Up Railway (Backend + Database)

1 Create Railway account at railway.app

Free tier includes \$5 credit monthly — enough for a hackathon. No credit card needed.

2 Create new project → Add PostgreSQL service

Railway gives you a `DATABASE_URL` env var automatically.

3 Deploy Node.js server from GitHub

Connect your GitHub repo → select `/server` as root directory → Railway auto-detects Node.js.

4 Set environment variables in Railway

```
DATABASE_URL=postgres://... (auto-set)
JWT_SECRET=your_random_32_char_secret
NODE_ENV=production
SMTP_HOST=smtp.mailtrap.io
SMTP_PORT=587
```

```
SMTP_USER=your_mailtrap_user
SMTP_PASS=your_mailtrap_pass
CLIENT_URL=https://goalsynce.vercel.app
```

5 Run migrations

In Railway console: `psql $DATABASE_URL -f migrations/001_create_users.sql` for each file in order. Or write a `npm run migrate` script that runs them all.

Step 2: Deploy Frontend to Vercel

6 vercel.com → New Project → Import GitHub repo

Set root directory to `/client`. Framework: Vite. Build command: `npm run build`. Output: `dist`.

7 Set environment variable in Vercel

```
VITE_API_URL=https://goalsynce-api.railway.app/api
```

14. Recommended Coding Order

Follow this exact order to avoid integration blockers. Each phase is testable before moving on.

#	Phase	What to Build	Test
1	Database	Run all 10 migration SQL files. Seed demo data.	Can query tables in psql
2	Backend: Auth	<code>app.js</code> , <code>db.js</code> , <code>auth.routes.js</code> , <code>auth.controller.js</code> , <code>authenticate.js</code> , <code>authorize.js</code>	POST <code>/api/auth/login</code> returns JWT

3	Backend: Goals	<code>goals.routes.js</code> , <code>goals.controller.js</code> , <code>goal.service.js</code>	Can CRUD goals, submit validates weightage
4	Backend: Manager	<code>manager.routes.js</code> , <code>manager.controller.js</code>	Can approve/rework goals, locks correctly
5	Backend: Achievements	<code>achievements.routes.js</code> , <code>achievements.controller.js</code> , <code>score.service.js</code>	Can log actuals, score calculates correctly
6	Backend: Admin	<code>admin.routes.js</code> , <code>admin.controller.js</code> , audit service	Can create cycles, push shared goals, unlock
7	Backend: Reports	<code>reports.routes.js</code> , Excel/CSV export	Downloads a working .xlsx file
8	Frontend: Core	<code>App.jsx</code> , <code>AuthContext.jsx</code> , <code>api.js</code> , <code>Login.jsx</code> , <code>ProtectedRoute.jsx</code>	Can login, JWT persists on refresh
9	Frontend: Employee	Dashboard, GoalsPage, GoalForm, WeightageGauge, CheckinPage	Full employee journey works
10	Frontend: Manager	TeamDashboard, ApprovalQueue, CheckinView	Full manager journey works
11	Frontend: Admin	AdminDashboard, CycleManager, UserManager, Reports	Full admin journey works
12	Bonus: Email	<code>email.service.js</code> , trigger in controllers	Mailtrap receives emails
13	Bonus: Escalation	<code>escalation.job.js</code> , <code>escalation.service.js</code>	Cron runs, logs to DB
14	Bonus: Analytics	Analytics charts in Admin dashboard	Charts render with real data
15	Polish	Error boundaries, loading states, mobile responsiveness, README	Judge can demo without errors

15. Demo Seed Data

In `migrations/010_seed_demo_data.sql`, create these exact accounts for the judges:

Role	Email	Password	Name
Admin	admin@goalsynce.com	Admin@123	Priya Sharma (HR Head)
Manager	manager@goalsynce.com	Manager@123	Rahul Verma (L1 Manager)
Employee 1	employee@goalsynce.com	Employee@123	Aarav Singh
Employee 2	employee2@goalsynce.com	Employee@123	Meera Patel

Pre-seed data for the demo journey:

- 1. Active cycle: "FY 2025-26" with Q1 window currently open
- 2. 5 thrust areas: Sales Growth, Customer Experience, Operational Efficiency, People Development, Digital Transformation
- 3. Employee 1 (Aarav): 4 approved goals, Q1 actuals submitted, one check-in by manager
- 4. Employee 2 (Meera): 3 goals in submitted state (waiting for manager approval — lets judge see approval flow)
- 5. One shared goal pushed to both employees from admin
- 6. 2 audit log entries showing goal unlock history

16. Coherence Checklist — Before Final Demo

Run through this list before submitting. Every item must pass.

API ↔ Frontend Contract

- ☐ Every API response uses `{ success, data, message }` format
- ☐ Frontend services always access `response.data.data` (not `response.data`)
- ☐ All dates sent as ISO strings, parsed with `date-fns parseISO()` on frontend
- ☐ UoM values: frontend always sends lowercase (`'min'`, `'max'`), matches DB CHECK constraint

Business Logic

- ☐ Weightage validation runs on BOTH client (real-time) and server (on submit)
- ☐ Score formula is identical in `score.service.js` and `client/utils/scoreCalc.js`
- ☐ Locked goals return 403 if employee tries to edit via API directly
- ☐ Check-in window validation: server rejects achievement POST if window is closed
- ☐ Shared goal: child goal's weightage is independent, title/target are read-only

Role Security

- ☐ Manager can only see employees whose `manager_id = manager.id`
- ☐ Employee GET `/api/goals` always filtered by `employee_id = req.user.id`
- ☐ Admin routes use `authorize('admin')` middleware — test with employee token → gets 403
- ☐ No endpoint returns another employee's data to an employee role

Demo Quality

- ☐ All 3 demo login credentials work
- ☐ No console errors in browser
- ☐ Loading states show while API calls are in flight
- ☐ Error messages are human-readable (not "undefined is not a function")
- ☐ Export downloads a real file
- ☐ Mobile responsive (at least usable on a phone — judges may check)
- ☐ README has: live URL, repo link, architecture diagram, credential table

You're Ready to Code

Start with the database migrations, then the auth backend, then build outward. Each phase in the coding order is independently testable with Postman/Bruno before you touch the frontend. The entire system is designed to come together cleanly because every layer uses the same data shapes.

Good luck at AtomQuest Hackathon 1.0. This manual gives you everything you need to win. 

GoalSync · Build Manual v1.0 · Generated for AtomQuest Hackathon 1.0